Week 9: AI Infrastructure & Compute Exploitation

ShadowRay, LLMjacking, Framework Vulnerabilities, and Vector DB Manipulation

AIM 5014: Special Topics in AI Safety & Security

Katz School of Science and Health
Yeshiva University

Fall 2026

Roadmap for Today

1. Module I: Unauthenticated AI Compute Hijacking (ShadowRay)
2. Module II: Exposed Model Registries API Keys (LLMjacking)
3. Module III: Orchestration Framework Exploitation
4. Module IV: Vector Database Exploitation
5. Module V: Synthesis Mitigation Strategies

The AI Compute Infrastructure Shift

Scaling Beyond Traditional Containers

Modern LLM serving, fine-tuning, and inference pipelines rely on specialized distributed frameworks (e.g., Ray, vLLM, Triton Inference Server).

- **Distributed Coordination**: Frameworks like Ray coordinate massive clusters of GPUs across cloud providers to handle high-throughput workloads.
- **Speed over Security**: In the race to deploy AI features, clusters are frequently launched with default configurations, lacking authentication or network segmentation.

Case Study: The ShadowRay Attack (Oligo Security, 2024)

Active Exploitation of AI Workloads

ShadowRay was the first major documented campaign targeting unauthenticated Ray clusters in the wild.

- **The Vector**: Attackers scanned public IP spaces for exposed Ray dashboard ports (default port 8265) lacking authentication.
- **Remote Code Execution**: Using Ray's job submission API, attackers executed arbitrary Python code inside cluster worker nodes.
- **Cryptomining Exfiltration**: Compromised GPU clusters were hijacked for cryptomining or utilized to pivot deeper into enterprise cloud VPCs.

vLLM and Serving Framework Exposures

Exposing High-Performance Inference Engines

High-throughput serving engines like vLLM and Triton are optimized for latency, often omitting default request-signing or fine-grained access control.

Attack Scenarios:

- Unauthorized model inference draining GPU quotas.
- Model extraction via repeated high-volume prompting.

Infrastructure Risks:

- Side-channel leakage and denial-of-service (DoS) targeting shared GPU memory.

Exercise Implementation: Securing AI Clusters

Interactive Exercise

Scenario: An enterprise deploys an unauthenticated Ray cluster on Kubernetes for fine-tuning LLMs. External attackers discover port 8265 and execute malicious jobs. How do we secure this?

Implementation Guidance (Cluster Hardening):

- Enable mTLS and Token-based authentication in Ray configuration files (`RAY_USE_TLS=1`, `RAY_AUTH_TOKEN`).
- Enforce Kubernetes NetworkPolicies to block external ingress to internal cluster management ports.

LLMjacking: Stealing Cloud API Credentials

Financial and Operational Theft (Sysdig, 2024)

Attackers routinely scan public code repositories, container registries, and dataset dumps for hardcoded LLM API keys (OpenAI, Anthropic, Cohere) and cloud credentials.

- **The Profit Motive**: Rather than stealing data, attackers use stolen API keys to route heavy commercial LLM workloads (generation, coding assistance) through the victim's billing account.
- **Stealthy Persistence**: Attackers create hidden secondary API keys within compromised cloud provider dashboards to maintain persistent access.

Truffle Security (2025): Keys in AI Training Data

The Dataset Leakage Vector

Public AI training datasets (such as Hugging Face snapshots or Common Crawl derivatives) frequently contain unredacted secrets.

Key Findings:

- Over 12,000 active API keys, database passwords, and private SSH keys discovered inside open training datasets.

Root Cause:

- Developers scraping GitHub or internal wikis to build fine-tuning datasets accidentally included configuration files containing secrets.

Exposed Private Registries Container Risks

Misconfigured Container Hubs (Trend Micro)

Unprotected Docker registries and private MLflow model stores expose proprietary model weights and training code.

- **Intellectual Property Theft**: Competitors or nation-state actors download proprietary fine-tuned model weights directly from exposed registries.
- **Malware Injection**: Attackers replace legitimate base container images with trojanized versions containing backdoors.

Exercise Implementation: Secret Management

Interactive Exercise

Scenario: A data scientist hardcodes an OpenAI API key inside a Jupyter notebook used for fine-tuning, which is later pushed to a public GitHub repo. How do we remediate and prevent this?

Implementation Guidance (Secret Scanning Vaults):

- Immediately revoke and rotate the exposed API key in the provider dashboard.
- Implement pre-commit hooks (e.g., `trufflehog` or `git-guardian`) to block commits containing secrets. Store secrets in HashiCorp Vault or AWS Secrets Manager.

Orchestration Framework Exploitation

Targeting the Agent Glue

Frameworks like LangChain, LlamaIndex, and Kubeflow handle complex tool routing, memory management, and data ingestion, introducing severe systemic vulnerabilities.

- **Arbitrary Code Execution**: Many orchestration tools feature built-in Python REPL evaluation tools that can be hijacked via prompt injection.
- **Unsafe Deserialization**: Loading untrusted agent state files or configuration graphs can trigger RCE.

Case Study: LangChain Vulnerability (CVE-2025-68664)

Remote Code Execution via Agent Tools

Recent advisories (Red Hat Security, 2025) highlighted critical code execution flaws in popular LLM orchestration packages.

The Exploit Flow:

- An attacker submits an indirect prompt injection via a web page read by a LangChain agent.

Tool Hijacking

The injection exploits unsafe evaluation functions within helper tools, granting the attacker shell access to the host server.

ShadowMQ: Code Reuse in the AI Ecosystem (Gourion et al., 2025)

Systemic Propagation of Flaws

Gourion et al. (2025) demonstrated how widespread code reuse across AI orchestration libraries caused critical vulnerabilities to spread systemically.

- **The Pattern**: A vulnerable message-queue or networking utility written for one agent framework is copied into dozens of downstream open-source repositories.
- **Ecosystem Impact**: A single zero-day vulnerability in a utility module compromises thousands of distinct AI applications simultaneously.

Exercise Implementation: Sandboxing Orchestration Tools

Interactive Exercise

Scenario: A LangChain agent uses a Python REPL tool to calculate math problems for users. An injection tricks the agent into running `import os; os.system("rm -rf /")`. How do we secure this tool?

Implementation Guidance (Restricted Execution):

- Never expose raw Python REPL or shell tools to LLM agents unless running inside a heavily restricted, ephemeral container sandbox (e.g., restricted AST evaluators or restricted Docker containers).

Vector Database Exploitation (OWASP GenAI LLM08)

Manipulating RAG Memory

Vector databases store enterprise knowledge as high-dimensional embeddings. Unauthorized manipulation of these embeddings undermines the integrity of RAG applications.

- **Embedding Injection (Poisoning)**: Attackers insert malicious documents into the vector store so that specific user queries always retrieve attacker-controlled context.
- **Similarity Score Manipulation**: Crafting embeddings that artificially dominate cosine-similarity rankings, crowding out legitimate enterprise knowledge.

Unauthorized Manipulation of Vector Search

Compromising the Retrieval Layer

If access controls on vector databases are misconfigured, attackers can directly modify, delete, or inject embeddings.

Data Poisoning via RAG:

- Injecting false policy documents into corporate search indices to trick customer support agents.

Denial of Service:

- Flooding vector indices with high-dimensional garbage to cause indexing timeouts and latency spikes.

Exercise Implementation: Securing Vector Databases

Interactive Exercise

Scenario: A multi-tenant enterprise RAG application allows any authenticated user to write documents into a shared Pinecone vector index, permitting users to overwrite competitor documents. How do we fix this?

Implementation Guidance (Access Control Metadata Filtering):

- Enforce strict multi-tenant namespace isolation in the vector database.
- Mandate cryptographically signed document ingestion pipelines and mandatory metadata access control tags for all retrieval queries.

Defense-in-Depth for AI Infrastructure

Securing the Entire AI Pipeline

Protecting AI infrastructure requires hardening every layer from compute clusters to retrieval databases:

1. **Compute Layer**: Authenticate all Ray/vLLM clusters with mTLS and tokens; restrict network ingress.
2. **Credential Layer**: Use automated secret scanners and vault management; eliminate hardcoded API keys.
3. **Framework Storage Layer**: Pin dependencies, use `safetensors`, and enforce strict vector DB access controls.

Discussion Prompt for Class

"If distributed compute frameworks like Ray prioritize cluster performance over default authentication, how can platform engineers balance speed of innovation with infrastructure security?"

Week 9 References & Further Reading (Part 1)

- Oligo Security (2024). ShadowRay Attack: AI Workloads Actively Exploited in the Wild.
- Sysdig (2024). LLMjacking: Financial Impact of Stolen Cloud Credentials in Generative AI.
- Trend Micro (2024). Exposed Private Registries Lead to Data Leaks and Malware.
- Truffle Security (2025). 12,000 API Keys and Passwords Exposed in AI Training Data.
- Red Hat Security (2025). LangChain Vulnerability Advisory (CVE-2025-68664).

Week 9 References & Further Reading (Part 2)

- Gourion et al. (2025). ShadowMQ: How Code Reuse Spread Critical Vulnerabilities Across the AI Ecosystem. *Oligo Security*.
- OWASP GenAI (2025). Vector and Embedding Weaknesses (LLM08).
- Lumelsky et al. (2024). ShadowRay: Security Analysis of Distributed AI Clusters.

Review: Unauthenticated Ray Clusters

Cluster Exposure

Exposed Ray dashboard ports permit unauthenticated remote code execution.

- Attackers hijack GPUs for cryptomining or cloud pivoting.

Review: The LLMjacking Threat

Billing Theft

Stolen API keys route heavy enterprise workloads through victim billing accounts.

- Requires continuous secret scanning across code and dataset repositories.

Review: Secrets in Training Data

Dataset Hygiene

Open training snapshots frequently contain unredacted API keys and passwords.

- Developers must scrub datasets before public release.

Review: Orchestration Tool Risks

Agent Glue Vulnerabilities

LangChain and LlamaIndex tools like Python REPLs can be hijacked via prompt injection.

- Demands strict containerized execution sandboxes.

Review: Code Reuse Propagation

ShadowMQ Effect

Vulnerable utility modules spread rapidly across open-source AI frameworks.

- Highlights the danger of unvetted library copying in the AI ecosystem.

Review: Vector Database Poisoning

RAG Integrity

Unauthorized embedding manipulation corrupts retrieval results and enterprise knowledge.

- Requires multi-tenant namespace isolation and signed ingestion pipelines.

Review: Container Registry Security

Private Hub Hardening

Exposed private registries allow IP theft and malware injection.

- Mandates strict role-based access control (RBAC) and image scanning.

Review: The Principle of Least Privilege

Agent Permissions

AI agents and tools should operate with minimal necessary privileges.

- Limits blast radius when a tool or dependency is compromised.

Review: Automated Dependency Scanning

CI/CD Security

Integrating Snyk, `pip-audit`, and git guardians into build pipelines.

- Catches slopsquatting and compromised dependencies before deployment.

Conclusion: Hardening AI Infrastructure

Final Security Posture

Securing AI requires looking beyond model weights to protect the underlying infrastructure, orchestration layers, and data pipelines.

Week 10: Model Inversion & IP Theft (Model Extraction)

- *Black-Box Model Theft via API Queries:* Reverse-engineering model parameters.
- *Training Data Extraction:* Reconstructing verbatim memorized PII or copyrighted text.
- *System Prompt Extraction:* Extracting hidden instructions and guardrail logic.

Required Reading/Viewing

- Nasr et al. 2023: Scalable Extraction of Training Data from Production Language Models
- Carlini et al. 2024: Stealing Production Language Models via Logit Queries
- LouisShark et al. 2024: Collection of Extracted System Prompts